Machine Learning - Assignment 2

M.Tech (AIML) | Work Integrated Learning Programmes Division | BITS Pilani

Project: Online Shopper Purchase Prediction

Name: Amarjith CK | **Student ID:** 2026AJ05064

1. GitHub Repository Link

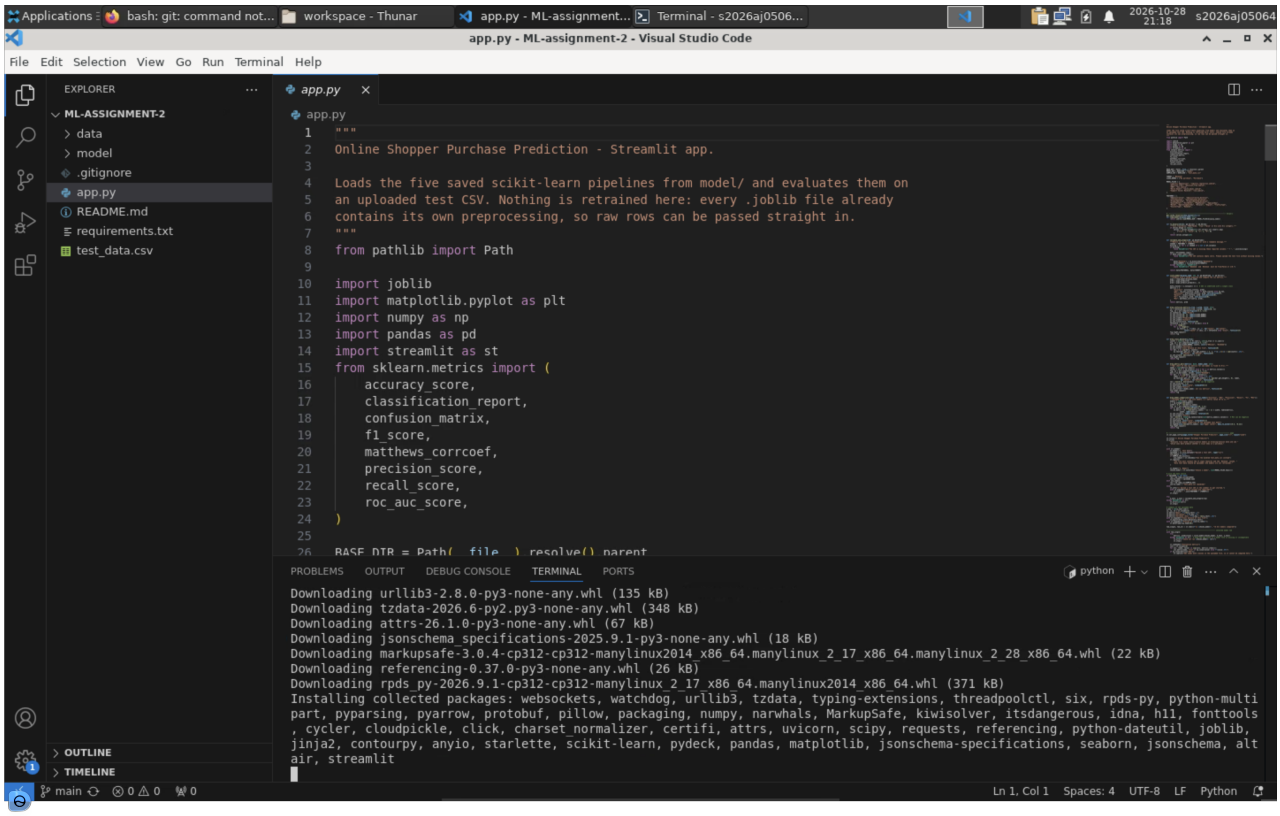
https://github.com/Amarjith-bits/ML-assignment-2

2. Live Streamlit App Link

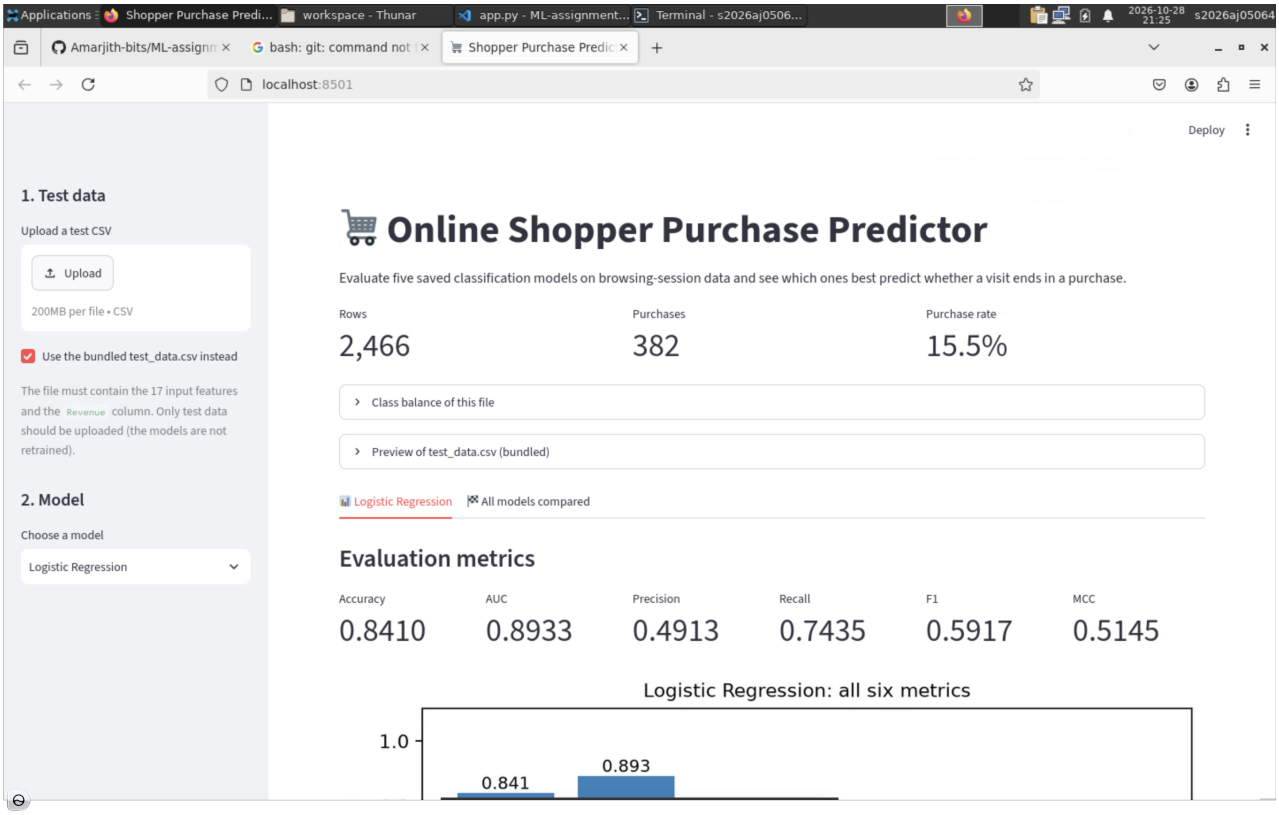
https://ml-assignment-2-qmweakzn4xczxouvufzvkf.streamlit.app

Deployed on Streamlit Community Cloud.

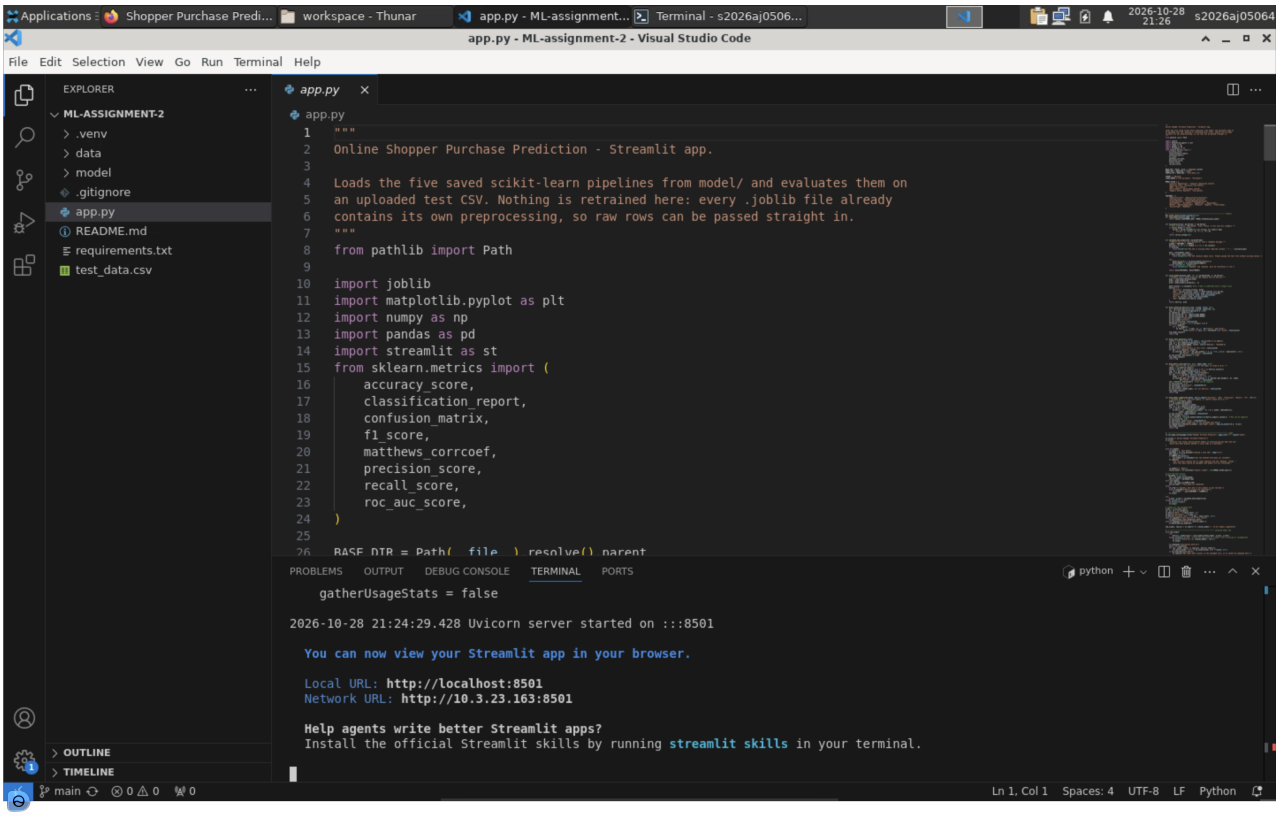
3. Screenshots: Execution on BITS Virtual Lab



Screenshot 1: Project open in VS Code on the BITS Virtual Lab, installing the dependencies from requirements.txt in the terminal.



Screenshot 2: The Streamlit app running on the BITS Virtual Lab (localhost:8501) with the saved models evaluated on test_data.csv.



Screenshot 3: Terminal on the BITS Virtual Lab showing the Streamlit server started on port 8501.

4. README Content

Content of README.md from the GitHub repository.

Online Shopper Purchase Prediction

Machine Learning – Assignment 2 (M.Tech AIML, BITS Pilani WILP)

---

a. Problem Statement

An online store sees thousands of browsing sessions, and most of them end without a purchase. This project builds and compares five classification models that predict, from a session's browsing behaviour and visitor attributes, whether the session will end in a purchase (`Revenue` = True) or not (`Revenue` = False).

This is a **binary classification** problem with a **class imbalance**: only about 15.5% of sessions end in a purchase. Because of this, accuracy alone is misleading (a model that always predicts "no purchase" would score about 84.5%), so the models are compared on six metrics: Accuracy, AUC, Precision, Recall, F1 and MCC.

A model like this could help a store decide when to show a discount, a chat prompt or a reminder to a visitor who looks likely to buy or likely to leave.

The five models are deployed in a Streamlit web app where a held-out test CSV can be uploaded and evaluated with any saved model.

---

b. Dataset Description

Item	Details
Name	Online Shoppers Purchasing Intention Dataset
Source	UCI Machine Learning Repository (dataset ID 468): https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset
License	CC BY 4.0
Instances	12,330 browsing sessions
Features	17 input features (10 numerical, 7 categorical)
Target	Revenue (True = purchase, False = no purchase)
Class distribution	10,422 no purchase (84.5%), 1,908 purchase (15.5%)
Missing values	None
Problem type	Binary classification, imbalanced

Each row is one browsing session. According to the dataset description, each session belongs to a different user over a one-year period, so the rows are independent and the data is not a time series.

Features

Group	Features
Page visits	`Administrative`, `Informational`, `ProductRelated` (number of pages of each type visited)
Time spent	`Administrative_Duration`, `Informational_Duration`, `ProductRelated_Duration` (seconds on each page type)
Exit behaviour	`BounceRates`, `ExitRates`
Value	`PageValues` (average value of pages visited before a transaction)
Calendar	`SpecialDay` (closeness to a special occasion), `Month`, `Weekend`
Visitor and technical	`VisitorType`, `OperatingSystems`, `Browser`, `Region`, `TrafficType`

Preprocessing

- `Weekend` and `Revenue` converted from True/False to 1/0.
- Numeric features (including `Weekend`) are standardised with `StandardScaler` for Logistic Regression, KNN and SVM (and the Decision Tree, where it has no effect).
- `Month`, `VisitorType`, `OperatingSystems`, `Browser`, `Region` and `TrafficType` are one-hot encoded. The last four are integer IDs rather than quantities, so they are treated as categories.
- Naive Bayes uses its own preprocessing: Gaussian Naive Bayes on the numeric features only, because the many 0/1 one-hot columns violate its Gaussian assumption (see observations).
- Each model is saved as a single scikit-learn `Pipeline` (preprocessing + classifier), so the app can score raw uploaded rows without retraining.

Train/test split: 80/20 stratified random split with `random_state=42`: 9,864 training rows and 2,466 test rows. The test rows are saved in `test_data.csv`.

---

c. GitHub Repository Link

https://github.com/Amarjith-bits/ML-assignment-2

Live Streamlit app: **https://ml-assignment-2-qmweakzn4xczxouvufzvkf.streamlit.app**

---

d. Models Used

Models trained: Logistic Regression, Decision Tree, K-Nearest Neighbors, Gaussian Naive Bayes, and Support Vector Machine (RBF kernel). All five use the same train/test split.

Precision, Recall and F1 are calculated for the positive class (purchase). AUC is computed from the predicted probability of a purchase.

Settings: Logistic Regression (`max_iter=1000`, `class_weight="balanced"`); Decision Tree (`max_depth=6`, `min_samples_leaf=10`, `class_weight="balanced"`); KNN (`n_neighbors=7`); Gaussian Naive Bayes (numeric features only); SVM (`kernel="rbf"`, `probability=True`, `class_weight="balanced"`). KNN and Gaussian Naive Bayes have no class-weight option, so they are trained without balancing.

> **Note: why class balancing was used, and why not for every model.** Only about 15.5% of sessions are purchases, so models trained on the raw data lean towards predicting "no purchase". Without balancing, recall was low (Logistic Regression 0.359, SVM 0.466, Decision Tree 0.524), meaning many real buyers were missed. Setting `class_weight="balanced"` makes mistakes on purchase sessions count more during training. It raised recall to 0.744, 0.746 and 0.833 and improved F1 and MCC, at the cost of lower precision and accuracy. Balancing was applied to the training step only, so the test set keeps the real class distribution.
>
> KNN and Gaussian Naive Bayes have no `class_weight` option, and no resampling (over- or under-sampling) was used, so that every model is trained on exactly the same rows. They are therefore reported without balancing, which means the main comparison table is not perfectly like-for-like. The supplementary table below shows Logistic Regression, Decision Tree and SVM without balancing for reference; KNN and Naive Bayes are identical in both tables.

Comparison Table (evaluation on the 2,466-row test set)

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.8410	0.8933	0.4913	0.7435	0.5917	0.5145
Decision Tree	0.8491	0.9095	0.5080	0.8325	0.6310	0.5692
KNN	0.8747	0.7920	0.6834	0.3560	0.4682	0.4328
Naive Bayes	0.8410	0.8119	0.4876	0.5131	0.5000	0.4058
Support Vector Machine	0.8658	0.8953	0.5491	0.7461	0.6326	0.5625

Supplementary: same models without class balancing (KNN and Naive Bayes are unchanged)

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.8808	0.8830	0.7366	0.3586	0.4824	0.4592
Decision Tree	0.8958	0.9137	0.7273	0.5236	0.6088	0.5604
Support Vector Machine	0.8869	0.8829	0.7036	0.4660	0.5606	0.5127

Observations on Model Performance

ML Model Name	Observation about model performance
Logistic Regression	With balancing, recall reaches 0.744, so it finds about three in four buyers, but precision falls to 0.491 (about half its purchase alerts are false alarms). Without balancing, recall was only 0.359, with higher precision (0.737). Its AUC of 0.893 is good, which shows the ranking of sessions is sound and the default threshold was the main limitation. As a linear model it cannot easily capture threshold-style rules, which likely explains why it trails the Decision Tree on MCC (0.515 vs 0.569).
Decision Tree	The best model on MCC (0.569), AUC (0.910) and recall (0.833). Purchase behaviour likely depends on thresholds in features such as `PageValues` and `ExitRates`, which a tree captures directly. The depth limit (`max_depth=6`, `min_samples_leaf=10`) keeps it from memorising the training data. Its precision is modest (0.508), so it raises many false alarms in exchange for catching most buyers.
KNN	The weakest model on AUC (0.792) and among the weakest on MCC (0.433). Recall is low (0.356) while precision is high (0.683), so it rarely predicts a purchase. It has no class-weight option, so it was not balanced. A likely cause is that the one-hot columns add many dimensions and dilute the distance calculation, so the nearest neighbours are not very informative.
Naive Bayes	Lowest MCC (0.406) and F1 of only 0.500. Its accuracy (0.841) is slightly below the 84.5% obtained by always predicting "no purchase". In a first attempt using all features including the one-hot columns, it predicted a purchase for most sessions and reached only 0.27 accuracy; restricting it to the numeric features fixed this. A likely reason is that the independence and Gaussian assumptions fit poorly here, since `BounceRates` and `ExitRates` are strongly correlated and many features are skewed counts.
Support Vector Machine	Close second overall: highest F1 (0.633), AUC 0.895 and MCC 0.563, with a better precision/recall balance than the Decision Tree (precision 0.549, recall 0.746) and higher accuracy (0.866). The RBF kernel can model non-linear patterns. It is the slowest to train because `probability=True` runs internal cross-validation.
Overall, winner for this dataset?	**Decision Tree**, with the highest MCC (0.569), AUC (0.910) and recall (0.833). The Support Vector Machine is essentially tied (MCC 0.563, F1 0.633), and the gap is small enough that a different random split could reverse it, so the result comes from a single split. Class balancing improved F1 and MCC for Logistic Regression, Decision Tree and SVM, at the cost of precision and accuracy. MCC is the most reliable single metric here because of the class imbalance. The best choice also depends on the goal: if catching as many buyers as possible matters most, the Decision Tree is preferred (recall 0.833); if a better balance between false alarms and missed buyers is wanted, the SVM is a close alternative (F1 0.633, precision 0.549).

Methodology

1. The UCI CSV (12,330 sessions) was loaded, and `Weekend` and `Revenue` were converted from True/False to 1/0.
2. The data was split 80/20 with stratification on `Revenue` and `random_state=42`. A random split is appropriate because each session is an independent record with no time ordering, and stratifying keeps the 15.5% purchase rate in both the training and test sets.
3. Preprocessing (scaling and one-hot encoding) is bundled with each classifier in a scikit-learn `Pipeline` and fitted on the training set only, so no information from the test set leaks into training. Naive Bayes uses its own preprocessing (numeric features only).
4. Class balancing (`class_weight="balanced"`) was applied only when training the models that support it (Logistic Regression, Decision Tree, SVM). The test set keeps the original class distribution, so the reported metrics reflect realistic traffic.
5. Hyperparameters are fixed, sensible values. No hyperparameter tuning or cross-validation was done, and all results come from a single train/test split, so small differences between models should be read with caution.

6 All six metrics were computed on the held-out test set. Each fitted pipeline was saved with joblib, and the Streamlit app loads these files and evaluates them on an uploaded test CSV without retraining.

Streamlit App

The app (`app.py`) loads the saved models and does not retrain them. Features:

- Upload a test CSV (it must contain all input features and the `Revenue` column).
- Select one of the five models from a dropdown.
- View Accuracy, AUC, Precision, Recall, F1 and MCC for the selected model.
- View the confusion matrix and classification report.
- See a bar chart of all six metrics for the selected model.
- Compare all five models on the same uploaded file (table and grouped bar chart) in the "All models compared" tab.
- See the class balance (purchase vs. no purchase) of the uploaded file.

Project Structure

```
project-folder/
|-- app.py
|-- requirements.txt
|-- README.md
|-- test_data.csv
|-- data/
|   └── online_shoppers_intention.csv
|-- model/
|   ├── training/            (source code: common.py, one script per model, run_all.py)
|   ├── logistic_regression.joblib
|   ├── decision_tree.joblib
|   ├── knn.joblib
|   ├── naive_bayes.joblib
|   └── svm.joblib
```

How to Run

```
pip install -r requirements.txt

# train all five models, create test_data.csv and save the models
python model/training/run_all.py

# start the app locally
streamlit run app.py
```

Reference

Sakar, C.O., Polat, S.O., Katircioglu, M., Kastro, Y. (2019). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications*, 31, 6893–6908. Dataset: UCI Machine Learning Repository, ID 468.